

Summer Research Fellowship Programme (SRFP)

8-Week Internship Final Report

*A comprehensive report detailing work on:
Linux Environments, Digital Logic ICs (7447, 7474), Minor Projects
(Digital Clock, Calculator), and FPGA Implementation*

Prepared by:
Mr. Devang Sachin Gupta

Under the Supervision of:
Dr. G.V.V. Sharma

Indian Institute of Technology (IIT) Hyderabad
May 26, 2026 - July 20, 2026

Contents

1	Workspace & Environment Setup	3
1.1	Linux Dualboot Configuration	3
1.2	Command Line & Terminal Tools	3
1.2.1	Termux	3
1.2.2	Ranger	4
1.3	Text Editors and Typesetting	4
1.3.1	Neovim	4
1.3.2	LaTeX	5
2	Digital Logic Design & Components	6
2.1	Loop Constructs in Hardware Description	6
2.2	7474 D-Type Flip-Flop	6
2.2.1	Pin Configuration and Architecture	6
2.2.2	Functional Operation and Applications	7
2.3	7447 BCD to 7-Segment Decoder and Workflow	7
2.3.1	Hardware Implementation	7
2.3.2	Android Development and Upload Workflow	8
3	GATE Problem Analysis	10
3.1	Problem Statement (GATE IN 2022)	10
3.2	Systematic Solution	10
3.2.1	Truth Table	11
3.3	Hardware Implementation	12
3.3.1	Components Required	12
3.3.2	Circuit Connections	12
4	Minor Project: Digital Clock	13
4.1	Project Overview and Specifications	13
4.2	Theory of Operation	13
4.3	Phase 1: Breadboard Prototyping	14
4.4	Phase 2: In-System Programming (ISP)	15
4.5	Phase 3: Standalone Hardware Integration	15
4.6	Phase 4: Permanent Assembly (Soldering)	16
4.7	Phase 5: Custom 3D Printed Enclosure	17
5	Calculator Design & Implementation	18
5.1	Project Overview	18
5.2	Hardware Design and Interfacing	18
5.3	Software Architecture and Parsing	18
5.4	RK4 Mathematical Engine	19

6	FPGA Integration and Hardware Exploration	20
6.1	Overview of the Custom Architecture	20
6.2	Interfacing and Experimental Setups	20
6.3	Deployment & Testing Workflow	20
7	Conclusion	22

Chapter 1

Workspace & Environment Setup

The initial phase of the internship required establishing a robust, efficient, and reproducible development environment. A Unix-based workflow was chosen to ensure compatibility with various hardware design tools, compilers, and typesetting engines required for the SRFP tasks.

1.1 Linux Dualboot Configuration

To leverage the full potential of open-source development tools, a Linux dual-boot environment (Ubuntu) was configured alongside the existing Windows operating system. This involved shrinking the Windows partition to allocate dedicated unallocated space, creating a bootable USB drive using tools like Rufus, and configuring the GRUB bootloader to seamlessly switch between the operating systems at startup.

Partitioning and disk management were handled carefully to separate the root (/) directory, swap space, and home (/home) directory. Post-installation, the system drives and partitions were verified using standard Linux command-line utilities.

Listing 1.1: Useful Disk Management Commands

```
# Update the package index post-installation
sudo apt update && sudo apt upgrade -y

# List all block devices and their partitions
lsblk

# Check disk space usage on mounted filesystems
df -h
```

1.2 Command Line & Terminal Tools

Mastering the command-line interface was a key objective to improve workflow speed and automate repetitive tasks.

1.2.1 Termux

For on-the-go accessibility and mobile environment testing, Termux was installed on an Android device. Termux acts as a powerful terminal emulator that provides a base Linux environment, allowing for remote SSH access to the main workspace and minor script testing when away from the workstation.

Listing 1.2: Termux Setup and SSH Configuration

```
# Update Termux packages
pkg update && pkg upgrade

# Install OpenSSH for remote server access
pkg install openssh

# Generate an SSH key pair
ssh-keygen -t rsa -b 4096
```

1.2.2 Ranger

To navigate complex directory structures efficiently without relying on a graphical user interface (GUI), **Ranger**, a console file manager with VI key bindings, was integrated into the workflow. Ranger allows for quick file previews, rapid directory traversal, and seamless execution of shell commands from within the file manager.

Listing 1.3: Installing and Launching Ranger

```
# Install Ranger via APT
sudo apt install ranger

# Launch Ranger in the terminal
ranger
```

1.3 Text Editors and Typesetting

Standardizing the code editing and document formatting processes was crucial for maintaining consistency across hardware description files and report drafts.

1.3.1 Neovim

Neovim was selected as the primary text editor due to its extensibility, low latency, and highly customizable nature. By configuring the `init.lua` file, essential plugins for syntax highlighting, auto-completion, and file exploration were added. Using VI/Vim keybindings significantly reduced the reliance on a mouse, thereby increasing coding efficiency for languages like C, Verilog, and Python.

Listing 1.4: Neovim Installation and Configuration Setup

```
# Install Neovim
sudo apt install neovim

# Create the configuration directory and file
mkdir -p ~/.config/nvim
touch ~/.config/nvim/init.lua

# Open the configuration file in Neovim
nvim ~/.config/nvim/init.lua
```

1.3.2 LaTeX

For technical documentation and the creation of this final report, LaTeX was utilized. LaTeX provides unparalleled control over document structure, mathematical typesetting, and referencing. The `texlive-full` distribution was installed to ensure all necessary packages (such as `circuitikz` for digital logic diagrams and `amsmath` for equations) were locally available.

Listing 1.5: LaTeX Compilation Workflow

```
# Install the complete TeX Live distribution
sudo apt install texlive-full

# Compile a LaTeX document into a PDF
pdflatex srfp_report.tex

# Clean up auxiliary files generated during compilation
rm *.aux *.log *.toc *.out
```

Chapter 2

Digital Logic Design & Components

2.1 Loop Constructs in Hardware Description

In hardware design, loop constructs are essential for generating repetitive logic structures without manually instantiating each component. Using `for` loops or `generate` blocks allows designers to create scalable architectures, such as multi-bit adders or shift registers, leading to cleaner and more maintainable code.

2.2 7474 D-Type Flip-Flop

The 7474 IC is a high-speed Dual D-type positive-edge-triggered flip-flop, a fundamental building block in sequential digital logic. Unlike combinational logic (such as the 7447 decoder) where outputs depend solely on current inputs, the 7474 incorporates memory, allowing it to store state information between clock cycles.

2.2.1 Pin Configuration and Architecture

The IC contains two independent flip-flops (Flip-Flop 1 and Flip-Flop 2) packaged in a standard 14-pin DIP configuration. The IC is powered via VCC (Pin 14) and Ground (Pin 7). For a single flip-flop (e.g., Flip-Flop 1), the primary interface pins are defined as follows:

- **Data Input (D - Pin 2):** The logical state (0 or 1) presented at this pin is transferred to the output during the clock transition.
- **Clock (CLK - Pin 3):** A positive-going transition (LOW to HIGH edge) triggers the data transfer. The IC is strictly edge-triggered, meaning changes on the D pin while the clock is steady (HIGH or LOW) do not affect the output.
- **Outputs (Q - Pin 5 and $\overline{Q}$ - Pin 6):** Q represents the current stored state, while $\overline{Q}$ provides the complementary (inverted) state.
- **Asynchronous Inputs (PRESET - Pin 4 and CLEAR - Pin 1):** These active-low pins override both the clock and data inputs. Setting PRESET to LOW forces Q to HIGH, while setting CLEAR to LOW forces Q to LOW. For normal synchronous operation, both must be tied to HIGH (5V).

2.2.2 Functional Operation and Applications

The defining characteristic of the 7474 is its precise timing requirement. Data must be stable at the D input for a specific setup time prior to the rising edge of the clock signal, and held for a brief hold time afterward to ensure reliable state capture.

In the context of hardware design and the minor projects developed, the 7474 was instrumental in designing robust sequential circuits:

- **Frequency Division (T-Flip Flop Configuration):** By hardwiring the inverted output ($\overline{Q}$) back into the Data (D) input, the flip-flop toggles its state on every rising clock edge. This effectively divides the input clock frequency by a factor of two. Cascading multiple such setups forms the basis of binary counters and precise timing circuits.
- **Shift Registers and State Machines:** Cascading multiple 7474 flip-flops (connecting the Q output of one stage to the D input of the next) allows for the creation of shift registers. This is essential for serial-to-parallel data conversion and the implementation of finite state machines (FSM).
- **Debouncing Switches:** Mechanical switches often produce multiple noisy pulses when pressed. Using the flip-flops as debounce latches ensures clean, single-pulse signals are sent to downstream logic gates, preventing erratic counting or false triggers.

2.3 7447 BCD to 7-Segment Decoder and Workflow

For displaying numeric outputs, the 7447 BCD to 7-Segment Decoder was interfaced with a Common Anode 7-segment display.

2.3.1 Hardware Implementation

The physical circuit was built on a breadboard and interfaced with an Arduino Uno.

The pinout and wiring configuration are defined as follows:

- Arduino D2 connects to 7447 Pin 7 (A)
- Arduino D3 connects to 7447 Pin 1 (B)
- Arduino D4 connects to 7447 Pin 2 (C)
- Arduino D5 connects to 7447 Pin 6 (D)
- Arduino 5V connects to 7447 Pin 16
- Arduino GND connects to 7447 Pin 8
- 7447 Pin 3 connects to +5V

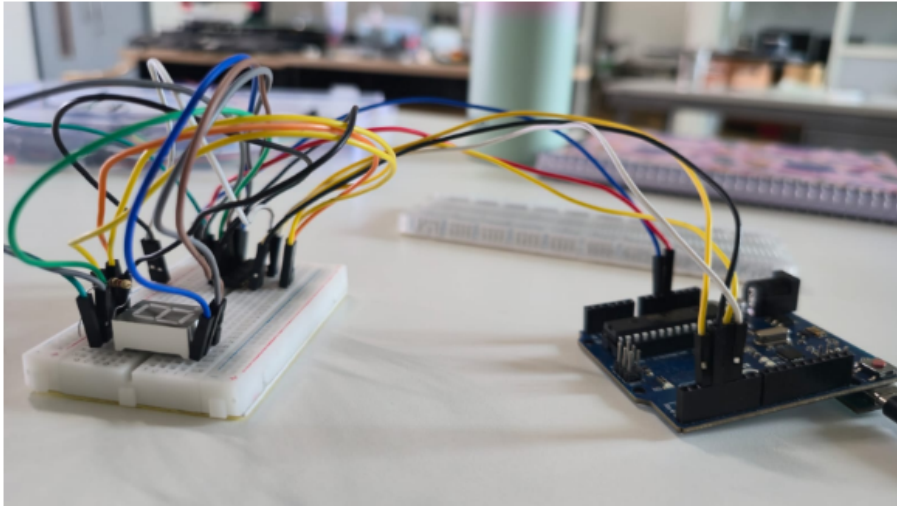


Figure 2.1: Breadboard setup of the 7447 IC and 7-segment display connected to Arduino Uno

- 7447 Pin 4 connects to +5V
- 7447 Pin 5 connects to +5V
- 7447 Pin 13 connects to Segment a
- 7447 Pin 12 connects to Segment b
- 7447 Pin 11 connects to Segment c
- 7447 Pin 10 connects to Segment d
- 7447 Pin 9 connects to Segment e
- 7447 Pin 15 connects to Segment f
- 7447 Pin 14 connects to Segment g
- Seven Segment Common Pins connect to +5V

2.3.2 Android Development and Upload Workflow

A unique aspect of this implementation was executing the entire development, compilation, and upload process on an Android device. The primary tools for this workflow include Termux, Ubuntu (via proot-distro), Neovim (nvim), Arduino CLI, and Arduino Droid.

Setup and Configuration: The initial environment requires installing packages like `proot-distro`, `git`, `curl`, `wget`, `neovim`, and `avra` within Termux. An Ubuntu container is then installed and accessed using `proot-distro install ubuntu` and `proot-distro login ubuntu`. Inside the Ubuntu container, Arduino CLI is downloaded, and the `arduino:avr` core is installed via `arduino-cli core install arduino:avr` to provide Arduino Uno support.

Development and Compilation: Source code is managed by downloading sketches directly from the GitHub repository using `curl -L <url> -o <filename>` into specific project directories (e.g., `~/Arduino Projects/inc_dec`). Editing is performed purely in the terminal using Neovim (`nvim filename.ino`). Once edited, sketches are compiled via the terminal command: `arduino-cli compile --fqbn arduino:avr:uno ..`. For AVR Assembly programs, the code is compiled using `avra filename.asm`, which generates the necessary HEX and OBJ files.

Hardware Upload: To flash the compiled code, the generated HEX file must be copied from the hidden `.cache` directory to the device's internal storage, specifically into the ArduinoDroid export folder using `cp *.hex /sdcard/ArduinoDroid/exp5/`. The Arduino Uno is connected to the Android device via an OTG cable. Finally, ArduinoDroid is used to select the precompiled HEX file and upload it directly to the board.

Chapter 3

GATE Problem Analysis

3.1 Problem Statement (GATE IN 2022)

The following problem was analyzed and simulated in hardware. The objective is to determine the Boolean expression for the output F of the logic block shown in Figure 3.1.

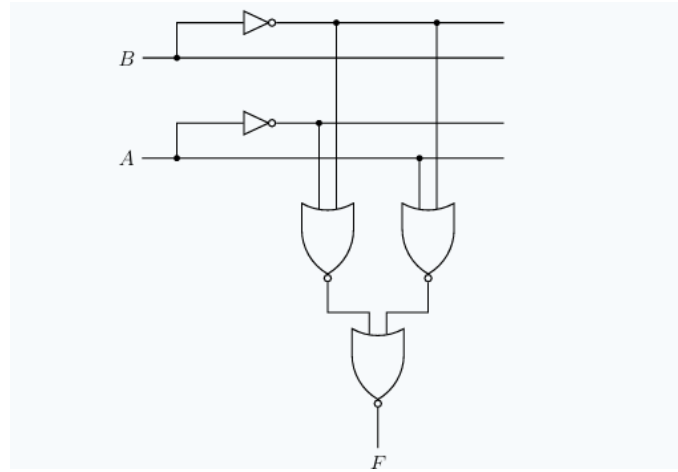


Figure 3.1: Logic circuit for GATE IN 2022 problem

The given options are:

- a) $A + B$
- b) $A\overline{B}$
- c) $A + \overline{B}$
- d) $\overline{B}$

3.2 Systematic Solution

Step 1: Signal Identification on the Main Buses The circuit generates inverted signals using the two NOT gates. Let's establish the signals on the four horizontal lines from top to bottom:

- Line 1 (Top): Output of the top NOT gate $\Rightarrow \overline{B}$
- Line 2: Direct input line for B
- Line 3: Output of the bottom NOT gate $\Rightarrow \overline{A}$

- Line 4 (Bottom): Direct input line for A

Step 2: Analysis of the First-Stage NOR Gates By tracing the vertical lines from the horizontal buses to the inputs of the two parallel NOR gates, we can derive their Boolean expressions:

- **Left NOR Gate (Y_1):** The inputs are connected to Line 3 ($\overline{A}$) and Line 1 ($\overline{B}$). Applying the NOR operation and De Morgan's Law:

$$Y_1 = \overline{\overline{A} + \overline{B}} = \overline{\overline{A}} \cdot \overline{\overline{B}} = A \cdot B \quad (3.1)$$

- **Right NOR Gate (Y_2):** The inputs are connected to Line 4 (A) and Line 1 ($\overline{B}$). Applying the NOR operation:

$$Y_2 = \overline{A + \overline{B}} = \overline{A} \cdot \overline{\overline{B}} = \overline{A} \cdot B \quad (3.2)$$

Step 3: Evaluating the Final Output (F) The final NOR gate evaluates the outputs Y_1 and Y_2 :

$$F = \overline{Y_1 + Y_2} = \overline{(AB) + (\overline{A}B)} \quad (3.3)$$

Factoring out the common term B :

$$F = \overline{B(A + \overline{A})} \quad (3.4)$$

According to Boolean algebra rules, the OR of a variable with its complement is always 1 ($A + \overline{A} = 1$):

$$F = \overline{B \cdot 1} = \overline{B} \quad (3.5)$$

Therefore, the correct option is **(d)** $\overline{B}$.

3.2.1 Truth Table

The theoretical findings are verified by the following truth table:

A	B	$Y_1 = AB$	$Y_2 = \overline{A}B$	$F = \overline{Y_1 + Y_2}$
0	0	0	0	1
0	1	0	1	0
1	0	0	0	1
1	1	0	1	0

Table 3.1: Truth Table verification for Output F

3.3 Hardware Implementation

3.3.1 Components Required

- 1x Arduino (Uno/Nano)
- 1x Breadboard
- 1x 7447 Decoder (BCD to 7-Segment)
- 1x Common Anode 7-Segment Display
- 7x 220Ω Resistors (Current limiting for the display segments)
- Jumper Wires

3.3.2 Circuit Connections

A. Decoder (7447) to Arduino (Inputs): Since the logic circuit's final output F evaluates to only a single bit (0 or 1), we exclusively control Input A (the Least Significant Bit) of the 7447 decoder.

- 7447 Input A (Pin 7): Connect to Arduino Pin 4 (This pin carries computed Output F).
- 7447 Inputs B (Pin 1), C (Pin 2), D (Pin 6): Connect directly to GND.
- 7447 LT / Pin 3 (Lamp Test): Connect to 5V.
- 7447 BI/RBO / Pin 4 (Blanking): Connect to 5V.
- 7447 RBI / Pin 5 (Ripple Blanking): Connect to 5V.
- 7447 VCC (Pin 16) & GND (Pin 8): Connect to 5V and GND respectively.

B. Decoder to 7-Segment Display (Outputs): Each output pin from the 7447 decoder connects to the corresponding segment of the display through a 220Ω resistor. The connections are: Pin 13 $\rightarrow$ Seg A, Pin 12 $\rightarrow$ Seg B, Pin 11 $\rightarrow$ Seg C, Pin 10 $\rightarrow$ Seg D, Pin 9 $\rightarrow$ Seg E, Pin 15 $\rightarrow$ Seg F, and Pin 14 $\rightarrow$ Seg G. The Display Common Pin connects directly to 5V.

Overview: The Arduino Uno computes the Boolean logic and sends the resulting single-bit output (F) from Pin 4 to the 7447 decoder's Input A. The decoder translates this binary signal into a 7-segment format, driving the common-anode display via 220Ω current-limiting resistors. The full automated Arduino C++ and Assembly code simulating this logic is available on the project's GitHub repository.

Chapter 4

Minor Project: Digital Clock

4.1 Project Overview and Specifications

This project documents the comprehensive design, prototyping, permanent assembly, and firmware implementation of a standalone digital clock. The system is engineered around the following core specifications:

- **Microcontroller:** ATmega328P-PU (Standalone bare chip)
- **Clock Source:** 16 MHz Quartz Crystal Oscillator
- **Display Type:** 4-Digit 7-Segment Display (Multiplexed)
- **Power Supply:** 5V DC via Type-B USB module

4.2 Theory of Operation

A digital clock functions as a highly precise counting machine. The architecture relies on several interconnected subsystems:

- **The Brain & Timing:** The ATmega328P utilizes a hardware timer to count the microscopic pulses from the 16 MHz crystal oscillator. Exactly 16,000,000 vibrations constitute one second. Two 22 pF capacitors are placed on pins 9 and 10 to Ground to act as shock absorbers, stabilizing the electrical resonance and preventing drift.
- **Power & Stability:** Two 100 nF ceramic bypass capacitors are placed directly across the power pins (7 & 8, and 20 & 22). These act as local energy reserves, filtering high-frequency noise and supplying instant current during rapid display multiplexing.
- **Logic Protection:** A 10 k Ω pull-up resistor is tied from 5V to Pin 1 (RESET) to securely hold the pin in a HIGH state, preventing random electromagnetic noise from spontaneously resetting the clock to 12:00.
- **Display Multiplexing:** To conserve GPIO pins, the 4-digit display rapidly cycles power to one digit at a time utilizing Persistence of Vision. Seven 330 Ω resistors strictly limit current to the segments (A-G), protecting both the microcontroller and the LEDs from thermal runaway.

4.3 Phase 1: Breadboard Prototyping

Initial testing was conducted using an Arduino Uno as the primary controller before migrating to a standalone chip.

- **Segment Wiring (The Shapes):** Arduino digital pins D2 through D8 were connected to display segments A through G, each with a 330Ω inline resistor.
- **Digit Selects (Multiplexing):** Arduino pins D9, D10, D11, and D12 were connected directly to the Digit 1 through Digit 4 common pins on the display.
- **Colon & Interface:** Two central red LEDs (colon) were driven by analog pin A3 with 330Ω resistors to Ground. Three push buttons (Hour, Minute, Mode) were connected to pins A0, A1, and A2. By utilizing the software INPUT_PULLUP function, the buttons only required a direct connection to Ground.

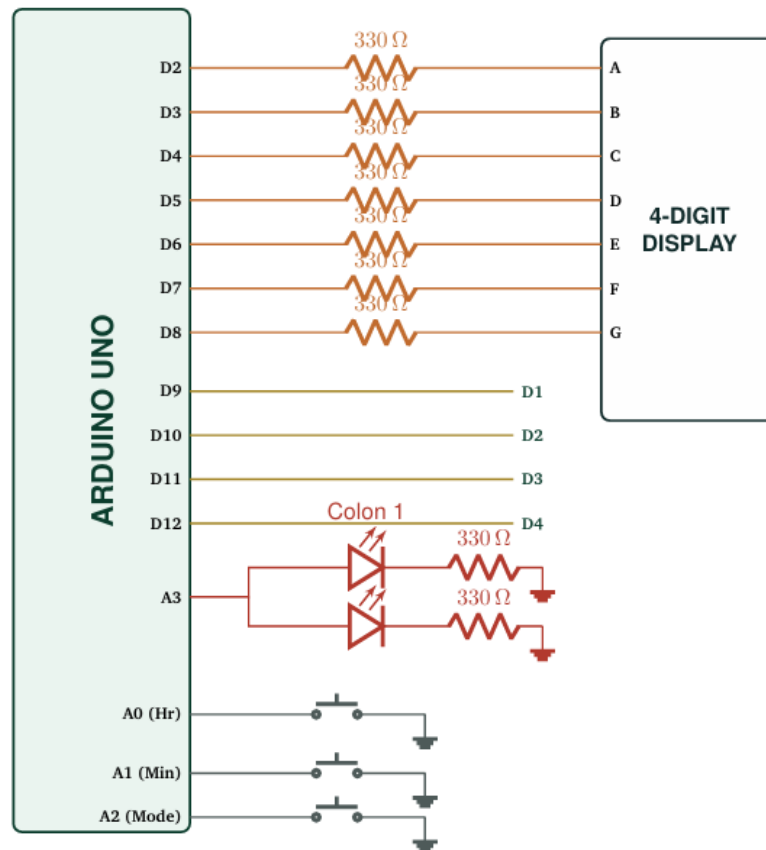


Figure 1: Detailed Electronic Schematic for Prototype Testing.

Figure 4.1: Detailed Electronic Schematic for Prototype Testing

4.4 Phase 2: In-System Programming (ISP)

To operate independently, the bare ATmega328P chip required a bootloader. The Arduino Uno was configured to act as a programmer.

1. **Target Life Support:** The standalone chip was powered via the Uno (5V to pins 7/20, GND to pins 8/22). The 16 MHz crystal and 22 pF capacitors were wired to pins 9 and 10, alongside the 10 k Ω reset pull-up resistor.
2. **SPI Data Lines:** The Uno's D10 was connected to the target's RESET (Pin 1). MOSI (D11) connected to target Pin 17, MISO (D12) to Pin 18, and SCK (D13) to Pin 19.
3. **Auto-Reset Prevention:** A 10 μ F capacitor was placed between the Uno's RESET and GND pins to prevent it from rebooting during the burn process.

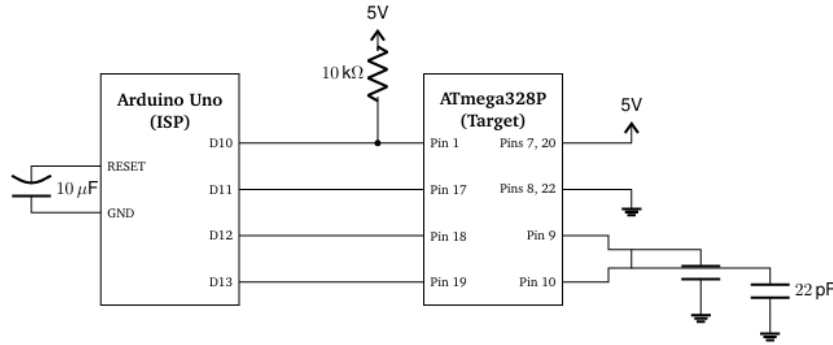


Figure 2: In-System Programming (ISP) Wiring Diagram.

Figure 4.2: In-System Programming (ISP) Wiring Diagram

4.5 Phase 3: Standalone Hardware Integration

Following the bootloader installation, the Uno was removed, and the ATmega328P was wired for independent execution.

- **Segments A-G:** IC Pins 4-6 and 11-14 (via 330 Ω resistors).
- **Digit Selects:** IC Pins 15-18 connected directly to the display digit pins.
- **Colon LEDs & Buttons:** IC Pin 26 controlled the colon LEDs, while Pins 23, 24, and 25 monitored the Hour, Minute, and Mode buttons respectively.

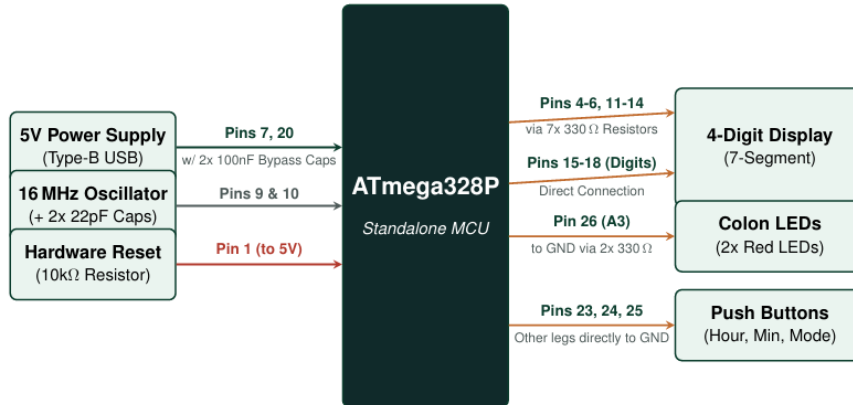


Figure 4.3: System Block Wiring Diagram for the Standalone Circuit

4.6 Phase 4: Permanent Assembly (Soldering)

The verified breadboard circuit was transferred to a perforated circuit board (perf-board).

1. **Component Placement:** A 28-pin DIP socket was centered on the board. The 16 MHz crystal and its 22 pF capacitors were placed as tightly as possible to pins 9 and 10 to prevent signal degradation. Figure 4 illustrates the component layout from the top view.
2. **Soldering Technique:** Components were tacked down by soldering opposite corners first. The snapped-off legs from the resistors were repurposed as solid-core wire to manually route the traces on the underside of the board, as shown in Figure 5.

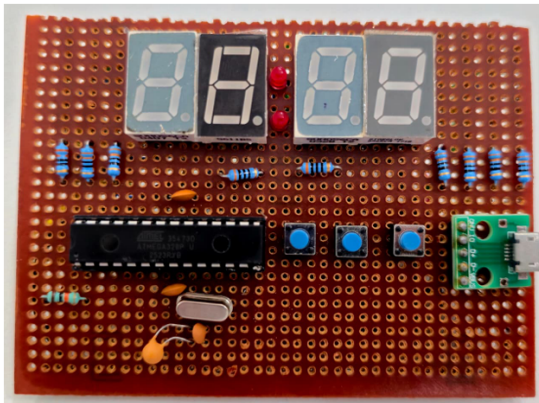


Figure 4.4: Recommended component layout on the dotboard

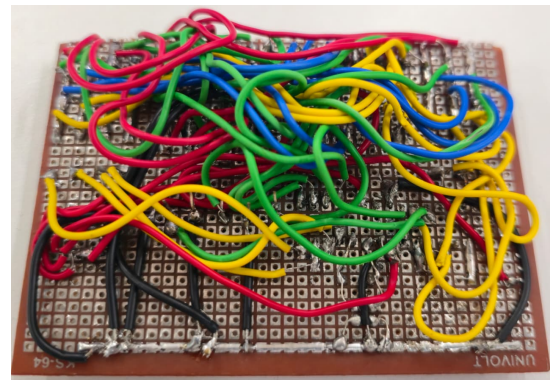


Figure 4.5: The fully soldered standalone clock back

4.7 Phase 5: Custom 3D Printed Enclosure

To secure the electronics without relying on mechanical fasteners or adhesives, a custom CAD enclosure was designed utilizing a mechanical "snap-fit" mechanism.

- **Design Mechanics:** The enclosure features a deep bottom tray and a top lid. Raised sections along the inner walls contain tiny horizontal lips. As the halves are pushed together, the thermoplastic flexes and snaps back, locking the case shut.
- **Fabrication:** PLA or PETG filaments are recommended with a fine layer height (0.16 mm to 0.20 mm). Both shells were printed flat on their backs to ensure the interlocking tabs were built horizontally for maximum strength.

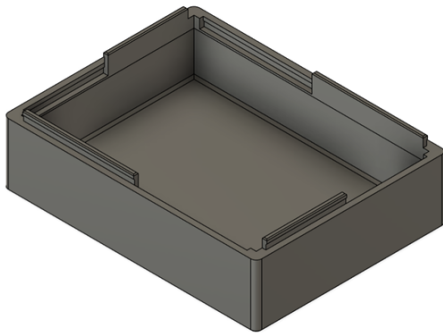


Figure 4.6: The deep bottom tray

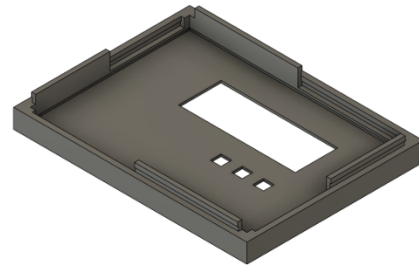


Figure 4.7: The top lid

Chapter 5

Calculator Design & Implementation

5.1 Project Overview

The objective of this project was to design a fully functional scientific calculator on an Arduino UNO (ATmega328P). A significant challenge was the absence of a hardware Floating Point Unit (FPU) on 8-bit AVR microcontrollers. To bypass standard, memory-heavy C math libraries, every mathematical function was implemented from first principles using numerical methods derived from Ordinary Differential Equation (ODE) models.

5.2 Hardware Design and Interfacing

The system hardware comprised the ATmega328P microcontroller running at 16 MHz, a 16×2 LCD (HD44780), and a custom 5×5 tactile push-button matrix.

- **Keypad Matrix:** To minimize GPIO usage, the 25 buttons were wired into a 5-row by 5-column matrix, requiring only 10 pins (D8-D12 for rows, A0-A4 for columns). The microcontroller scans input by driving each row LOW in turn and reading the columns.
- **LCD Output:** The display was connected in 4-bit mode to conserve pins, utilizing D4-D7 for the data bus, and D2 and D3 for Register Select and Enable respectively.

5.3 Software Architecture and Parsing

The firmware was written in modular Embedded C (AVR-GCC). The end-to-end workflow handles user input, parses the mathematical expression, evaluates it, and outputs the result.

- **Shunting Yard Algorithm:** User expressions (Infix notation) are parsed into Reverse Polish Notation (RPN) using the Shunting Yard algorithm, adhering to a strict operator precedence table.
- **Bug Resolution:** During development, a bug in the tokeniser (`pars.c`) was identified where multi-digit numbers (e.g., "18") were erroneously split into

separate single-digit tokens ("1" and "8"). This was resolved by modifying the token-accumulation loop to correctly read consecutive digit characters before pushing the complete number to the RPN evaluator.

5.4 RK4 Mathematical Engine

Instead of lookup tables, functions like $\sin(x)$, $\ln(x)$, and e^x were reformulated as initial value ODE problems.

- **Implementation:** The ODEs were solved using the Runge-Kutta 4th Order (RK4) method. For example, e^x was modeled as $y' - y = 0$ with $y(0) = 1$.
- **Performance vs. Standard Methods:** Five numerical methods were evaluated. RK4 was selected because it achieved the lowest error (RMS 3.44×10^{-7}) at the 32-bit hardware precision floor, guaranteeing a predictable fixed execution time of 192.3
- **Resource Optimization:** The unified RK4-based framework proved highly efficient, consuming only 12.3 KB of Flash (38%) and 350 bytes of SRAM (17

design, ALU concepts, I/O handling

Chapter 6

FPGA Integration and Hardware Exploration

6.1 Overview of the Custom Architecture

The final phase of the SRFP internship explored high-speed parallel processing using a custom FPGA development board engineered at IIT Hyderabad. This heterogeneous board integrates a versatile FPGA fabric—dedicated to deterministic, high-speed parallel execution—with an embedded ESP32 microcontroller that serves as the master control unit, handling Wi-Fi/Bluetooth connectivity and high-level state machines.

6.2 Interfacing and Experimental Setups

To evaluate the board’s capabilities and establish robust communication pipelines, practical integration tests were conducted using external development platforms:

- **Arduino UNO Integration:** Initial testing utilized an Arduino to establish basic I2C and SPI buses. Bi-directional logic level shifters were required to bridge the Arduino’s 5V logic with the FPGA’s 3.3V I/O banks. While successful for basic logic verification, the Arduino’s 16 MHz clock significantly bottlenecked the FPGA’s true parallel processing potential.
- **Raspberry Pi 3 Integration:** To achieve high-bandwidth data transfers, the setup was upgraded to a Raspberry Pi 3. Hardware SPI pins were connected directly to the FPGA (both operating safely at 3.3V) for megahertz-speed data pushing. In a parallel experiment, the on-board ESP32 was programmed to wirelessly publish processed data to a local Mosquitto MQTT broker hosted on the Pi, demonstrating a highly efficient edge-computing workflow.

6.3 Deployment & Testing Workflow

Working with this custom architecture required a standardized RTL design and deployment pipeline. Hardware architectures were drafted in Verilog, with pin constraints carefully synthesized to match the physical traces of the development board.

Once compiled, the bitstreams were successfully deployed using two methods: directly loading into the FPGA fabric via a dedicated JTAG programmer, or passing the bitstream through the ESP32, which successfully demonstrated the board's capability for remote Over-The-Air (OTA) hardware updates.

Chapter 7

Conclusion

The 8-week Summer Research Fellowship Programme (SRFP) at IIT Hyderabad, under the supervision of Dr. G.V.V. Sharma, has been a profoundly enriching experience that bridged the gap between theoretical computer science and practical hardware engineering.

The journey began with establishing a robust command-line development environment. Mastering Linux workflows, Termux, Neovim, and LaTeX laid a strong foundation for efficient coding and technical documentation. Progressing into digital logic, the hands-on implementation of the 7447 and 7474 ICs, along with the hardware realization of a GATE 2022 logic problem, solidified my understanding of Boolean algebra, combinational circuits, and sequential logic prototyping.

The internship culminated in the development of robust, standalone embedded systems. The Digital Clock project provided an end-to-end experience in microcontroller architecture—from burning bootloaders on bare ATmega328P chips (ISP) and permanent perfboard soldering, to designing and 3D printing custom mechanical snap-fit enclosures.

Similarly, the Scientific Calculator project pushed the architectural limits of 8-bit AVR microcontrollers. By implementing the Runge-Kutta 4th Order (RK4) numerical method from scratch, the project successfully bypassed the absence of a hardware Floating Point Unit (FPU). This demonstrated how efficient algorithmic design in Embedded C can overcome strict hardware constraints while maintaining high precision.

Finally, the exploration of the custom FPGA development board equipped with an embedded ESP32 SoC provided invaluable exposure to heterogeneous computing. Interfacing this board with platforms like the Arduino UNO and Raspberry Pi 3 highlighted the power of hardware-software co-design, high-speed SPI communication, and edge-computing IoT architectures.

In conclusion, this fellowship has immensely broadened my technical skillset, transforming fundamental theoretical concepts into tangible, optimized, and standalone hardware solutions. The rigorous debugging, system integration, and exposure to advanced research methodologies have thoroughly prepared me for future technical challenges in the domains of embedded systems and digital logic design.